

TÀI LIỆU THUYẾT MINH PHƯƠNG PHÁP

Hệ thống Multi-Agent Pipeline Giải Trắc Nghiệm Đa Lĩnh Vực

Dự thi: Cuộc thi Vietnamese Student HackAlthon 2026 – Bảng C (Innovator)

1. ĐẶT VẤN ĐỀ VÀ CHIẾN LƯỢC TỐI ƯU

1.1. Thách thức của Bảng C

Bảng C đặt ra yêu cầu cốt lõi: xây dựng một **AI Agent đa tác vụ** vận hành trên các mô hình ngôn ngữ nhỏ và vừa (SLMs/LLMs kích thước nhỏ như Gemma-4 Series hoặc Qwen3.5 với tham số lượng nhỏ hơn hoặc bằng 9B). Khi hoạt động trong điều kiện tài nguyên hạn chế (đặc biệt là khi bị lượng tử hóa về định dạng 4-bit), các mô hình này thường bộc lộ 3 hạn chế lớn:

- Ảo giác số học (Mathematical Hallucination):** Khả năng tính toán định lượng kém, dễ dẫn đến sai số nghiêm trọng trong các bài toán thuộc khối tự nhiên (Toán học, Vật lý, Hóa học).
- Nhiều định dạng đầu ra (Output Formatting Noise):** Khó tuân thủ tuyệt đối các ràng buộc cấu trúc nghiêm ngặt (ví dụ: chỉ trả về duy nhất một ký tự đáp án A, B, C, D) mà thường sinh thêm các đoạn giải thích dài dòng không cần thiết (*yapping*).
- Giới hạn cửa sổ ngữ cảnh (Context Window Constraints):** Khi xử lý các văn bản đọc hiểu có độ dài lớn, mô hình dễ dàng bỏ sót các chi tiết mang tính chất "bẫy" ngữ nghĩa (như các từ phủ định).

1.2. Chiến lược giải quyết

Để khắc phục triệt để các điểm nghẽn trên, nhóm chúng tôi đã xây dựng kiến trúc **Multi-Agent Graph**. Thay vì sử dụng một cấu trúc prompt cố định (Single-Prompt) để ép một mô hình duy nhất xử lý mọi tác vụ, hệ thống sẽ tự động định tuyến câu hỏi đến các Agent chuyên trách. Thiết kế này giúp tối ưu hóa đồng thời cả hai mục tiêu: **Độ chính xác cao nhất (chiếm 80 điểm)** và **Tốc độ suy luận tối ưu (chiếm 10 điểm)**.

2. KIẾN TRÚC HỆ THỐNG MULTI-AGENT PIPELINE

Luồng hoạt động của hệ thống được chia thành 4 giai đoạn logic chính, quản lý trạng thái tập trung và nhất quán thông qua biến toàn cục GraphState:

Giai đoạn 1: Phân luồng (Router Agent):

Mọi câu hỏi đầu vào đều được tiếp nhận bởi **Router Agent**. Agent này thực hiện phân tích nhanh ngữ cảnh và định tuyến chính xác câu hỏi vào một trong ba nhánh xử lý chuyên biệt:

- **FAST_QA:** Dành cho các câu hỏi khai thác tri thức nền tảng (Lịch sử, Địa lý,...).
- **READING:** Dành cho các bài toán đọc hiểu văn bản dài và phức tạp (Pháp luật, Văn học,...).
- **CODEABLE:** Dành cho các câu hỏi định lượng phức tạp cần tính toán số học (Toán, Lý, Hóa, Tài chính,...).

Giai đoạn 2: Trực tiếp trả lời (Direct Answering):

- **Fast-QA Agent:** Giải quyết các câu hỏi thuộc nhánh FAST_QA bằng phương pháp **Zero-Shot**. Để tối ưu hóa tốc độ phản hồi (Inference Time), Agent này được cấu hình tắt hoàn toàn tính năng suy luận dài dòng (sử dụng cờ /no_think), giúp trả kết quả ngay lập tức và tối đa hóa điểm số về mặt thời gian.
- **Reading Agent:** Chuyên trách nhánh READING. Agent được kích hoạt chế độ suy luận chuyên sâu (cờ /think) nhằm áp dụng cơ chế Chain-of-Thought (CoT). Điều này giúp mô hình phân tích kỹ lưỡng văn bản, chủ động phát hiện các bẫy phủ định hoặc thông tin ẩn khuất (như "không phải", "ngoại trừ").

Giai đoạn 3: Thực thi mã và Tự sửa lỗi (Coder & Sandbox Loop):

Đây là giải pháp cốt lõi của nhóm nhằm giải quyết triệt để điểm yếu tính toán ở các bài toán định lượng (CODEABLE):

1. **Code Generation:** LLM đóng vai trò là một lập trình viên, tự động viết một đoạn script Python thuần túy để giải phương trình hoặc xử lý số liệu bằng các thư viện chuyên dụng như sympy hoặc math thay vì tự tính toán thủ công.
2. **Sandbox Execution:** Đoạn mã Python được khởi chạy trong một môi trường cô lập an toàn (Sandbox) với chế độ ngắt tự động (Timeout = 5s) nhằm thu về kết quả số học chính xác tuyệt đối.
3. **Self-Correction (Tự sửa lỗi):** Nếu script phát sinh lỗi hệ thống (SyntaxError, RuntimeError) hoặc cho ra kết quả lệch quá 1% so với các phương án trắc nghiệm sẵn có, hệ thống sẽ nạp ngược log lỗi (stderr) để LLM tự động tối ưu và sửa mã nguồn (vòng lặp tối đa 2 lần).
4. **Majority Voting (Bỏ phiếu số đông):** Gộp kết quả từ Sandbox cùng 3 lượt chạy suy luận (reasoning runs) độc lập của LLM để thực hiện cơ chế bỏ phiếu, từ đó chốt lại đáp án có độ tin cậy cao nhất.

Giai đoạn 4: Chuẩn hóa đầu ra (Dynamic Mapper):

Sử dụng kết hợp giải pháp Rule-based bằng biểu thức chính quy (Regex) và độ đo tương đồng giao tập hợp (**Set Intersection Similarity**). Hàm xử lý này sẽ rà soát câu trả lời tự do của LLM (ví dụ: "*Đáp án đúng là A*"), loại bỏ hoàn toàn các từ dừng (stop-words) và trích xuất chính xác duy nhất một ký tự đại diện (A, B, C, hoặc D). Kỹ thuật này triệt tiêu hoàn toàn rủi ro mất điểm đáng tiếc do lỗi sai định dạng đầu ra.

3. TỐI ƯU HÓA MÔ HÌNH VÀ VẬN HÀNH DOCKER

3.1. Lựa chọn Mô hình

Hệ thống tối ưu hóa hiệu năng dựa trên nền tảng **google/gemma-4-E4B-it** được triển khai thông qua thư viện tối ưu **Unsloth**:

- **Lượng tử hóa 4-bit (4-bit Quantization):** Giảm đến 70% dung lượng VRAM tiêu thụ so với định dạng FP16 gốc, giúp hệ thống vận hành mượt mà trên các kiến trúc phần cứng giới hạn.
- **Bảo toàn năng lực:** Giữ vững khả năng lập luận logic và cấu trúc suy luận chuyên sâu gần như tương đương với phiên bản cấu hình đầy đủ.

3.2. Quản trị Bộ nhớ và Phòng chống Tràn cấu trúc (OOM Defense)

Để xử lý ổn định bộ dữ liệu Private Test quy mô lớn (2000 câu) từ Ban tổ chức mà không gặp sự cố sập hệ thống do tràn bộ nhớ (Out-of-Memory):

- **Micro-Batching:** Chia nhỏ luồng xử lý đầu vào thành các khối kích thước nhỏ (10 câu hỏi cho mỗi batch).
- **VRAM Cleanup:** Chủ động gọi hàm `torch.cuda.empty_cache()` và giải phóng các tensor kết quả (outputs) khỏi bộ nhớ ngay sau khi hoàn thành mỗi batch xử lý.
- **Checkpointing cơ chế động:** Trạng thái hệ thống (GraphState) liên tục được ghi dữ liệu (dump) xuống file cấu trúc `pipeline_checkpoint.json`. Trong trường hợp hệ thống xảy ra sự cố đột ngột hoặc container bị ngắt, lỗi hệ thống có thể tự động đọc checkpoint và tiếp tục tiến trình từ vị trí lỗi thay vì phải vận hành lại từ đầu.

4. TỔNG KẾT

Giải pháp Multi-Agent của nhóm chúng tôi không tiếp cận bài toán trắc nghiệm theo hướng "đoán mò" dựa trên xác suất thuần túy, mà cung cấp một công cụ suy luận có cấu trúc chặt chẽ: **Biết phân loại vấn đề, Biết tự sử dụng công cụ chuyên biệt, Biết tự kiểm chứng và sửa sai**. Kiến trúc này hoàn toàn đáp ứng xuất sắc bộ 3 tiêu chí cốt lõi của Bảng C: **Accuracy (Chính xác), Speed (Tốc độ xử lý) và Innovation (Tính sáng tạo)**.